

L03b · CORALNPU 教学仓

L03b · M 扩展、CSR 与异常

基础知识 —— 理论、术语与参考资料

对应上游仓库: [google/coralnpu](#) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-15 · 生成命令: `./learn pdf`

目录

第 1 章 M 扩展：乘除法

- 1.1 八条指令
- 1.2 位模式、值与解释
- 1.3 扩展与截断：两个不改变值的操作
- 1.4 为什么”高位乘法”必须先扩展再相乘
- 1.5 回到 Verilog：怎么把”先扩展”写出来
- 1.6 三个必须按规范处理的边界
- 1.7 组合实现 vs 多周期实现

第 2 章 CSR 与 Zicsr

- 2.1 CSR 是什么：机器的控制面板
- 2.2 CSR 都在管什么
- 2.3 CSR 地址不是随便编的：12 位里藏了两件事
- 2.4 Zicsr 六条指令
- 2.5 字段的读写规则：RO / WARL / WLRL
- 2.6 一次真实的异常：CSR 各司其职
- 2.7 本课实现的最小集合
- 2.8 CSR 也有数据冒险（本课踩过的坑）

第 3 章 异常：进入与返回

- 3.1 三个概念
- 3.2 完整的流程
- 3.3 常见的 mcause 取值
- 3.4 和上游的关系

第 4 章 自测题

第 5 章 参考资料

第 1 章 M 扩展：乘除法

1.1 八条指令

基础 RV32I 只有加减和移位，乘法要自己用移位加法拼。M 扩展补上硬件乘除法：

funct3	指令	语义	难点
000	<code>mul</code>	乘积低 32 位	有无符号结果相同
001	<code>mulh</code>	有符号 × 有符号 的高 32 位	两个操作数都符号扩展
010	<code>mulhsu</code>	有符号 × 无符号 的高 32 位	一个符号扩展、一个零扩展
011	<code>mulhu</code>	无符号 × 无符号 的高 32 位	都零扩展
100	<code>div</code>	有符号除法（向零取整）	除零、溢出
101	<code>divu</code>	无符号除法	除零
110	<code>rem</code>	有符号取余（符号跟被除数）	除零、溢出
111	<code>remu</code>	无符号取余	除零

表里 `h` 行末尾写的”符号扩展 / 零扩展”，不是”给乘法器换一套电路”的意思，而是**这条指令怎么解释两个 32 位寄存器里的位模式**。要读懂这一列，只需要 CS:APP（《深入理解计算机系统》第 3 版）第 2 章的五个小节：

你现在的疑问	去看 CS:APP
同一个位模式为什么会有两个数值？	2.2.2 无符号数编码、2.2.3 补码编码
<code>0xFFFF_FFFF</code> 到底是 -1 还是 4294967295？	2.2.3、2.2.4 有符号数与无符号数之间的转换
为什么零扩展、符号扩展”不改变值”？	2.2.6 扩展一个数字的位表示
丢掉高位（截断）到底做了什么？	2.2.7 截断数字
为什么 <code>mul</code> 不用区分有无符号？	2.3.4 无符号乘法、2.3.5 补码乘法（本节的核心）
真实 CPU 里也是两套乘法器吗？	3.5.5 特殊的算术操作（x86-64 的 <code>mulq / imulq</code> 产生 128 位乘积）

下面 1.2 ~ 1.5 节就是把这几节按”读懂 `mulh` 到底需要什么”重排了一遍。只用到加减乘和 2 的幂，没读过 CS:APP 也能跟上；原书那几节写得更细，值得对照着看第二遍。

1.2 位模式、值与解释

1.2.1 寄存器里没有”有符号数”

CS:APP 第 2 章反复强调的一件事：机器里只有**位模式**，“有符号/无符号”是我们给位模式挑的**解释方式**。同一个 32 位模式有两种标准解释（CS:APP 把它们记作 B2U 和 B2T）：

位模式	按无符号解释 B2U	按补码解释 B2T
0x0000_0002	2	2
0x7FFF_FFFF	2147483647	2147483647
0x8000_0000	2147483648	-2147483648
0xFFFF_FFFF	4294967295	-1

两种解释的换算公式只有一条，**本章后面所有推导都靠它**：

$$B2T(x) = B2U(x) - x_{31} * 2^{32} \quad // \text{ } x_{31} \text{ 是第 31 位（最高位），取 0 或 1}$$

也就是说：同一个位模式，按补码解释的值，等于按无符号解释的值**减去最高位的权重** 2^{32} 。
0xFFFF_FFFF 就是 $4294967295 - 2^{32} = -1$ 。

1.2.2 乘法指令名里的 s / u，是在挑选解释方式

于是 RISC-V 的 M 扩展必须把”用哪种解释”写进指令名：

- `mulh`：把两个操作数都按**补码**解释成值，相乘，取高位；
- `mulhu`：把两个操作数都按**无符号**解释成值，相乘，取高位；
- `mulhsu`：第一个按补码、第二个按无符号解释，相乘，取高位。

例：a = 0xFFFF_FFFF、b = 0x8000_0000，同一对位模式在三条指令下乘的是三个不同的数：

指令	实际相乘的两个数	数学乘积
<code>mulh</code>	$-1 \times (-2147483648)$	2147483648
<code>mulhsu</code>	-1×2147483648	-2147483648
<code>mulhu</code>	$4294967295 \times 2147483648$	9223372034707292160

结论：乘法指令真正要做的第一件事，是把位模式还原成”这条指令认为的那个值”。

1.3 扩展与截断：两个不改变值的操作

要把 32 位的位模式送进 64 位宽的乘法器，就必须先”搬家”；搬家的规则就是 CS:APP 2.2.6 / 2.2.7 的扩展与截断：

① **零扩展（zero extension）**：高位补 0，配无符号解释。

```
0xFFFF_FFFF --零扩展--> 0x0000_0000_FFFF_FFFF
B2U(前) = 4294967295, B2U(后) = 4294967295 // 值没变
```

② **符号扩展 (sign extension)**：把最高位复制到新增的每一位，配补码解释。

```
0xFFFF_FFFF --符号扩展--> 0xFFFF_FFFF_FFFF_FFFF
B2T(前) = -1, B2T(后) = -1 // 值没变
```

这就是 CS:APP 2.2.6 的两条结论：**零扩展保持无符号值不变，符号扩展保持补码值不变**。注意它们的适用范围是分开的：对 `0xFFFF_FFFF` 这个位模式，零扩展得到 4294967295，符号扩展得到 -1，**只有一种能把你要的那个值搬过去**。

③ **截断 (truncation)**：只留低 k 位，等价于对 2^k 取模 (CS:APP 2.2.7)。

```
0x1_FFFF_FFFE --截断到 32 位--> 0xFFFF_FFFE
```

截断丢的是”高位信息”，而且截断之后的位模式还可以用两种方式解释——这正是后面 `mul` 与 `mulh` 差别那么大的根源。

1.4 为什么”高位乘法”必须先扩展再相乘

1.4.1 一句话版本

乘法的结果，是两个”值”的乘积；而 32 位寄存器里只有位模式。“先扩展”就是把 32 位的位模式搬进 64 位、并且保证**值不变**（符号扩展或零扩展）；“再相乘”就是让乘法真的在 64 位宽度上发生，这样乘积的高 32 位才会被算出来。

反过来最容易理解为什么不能省这一步：如果乘法在 32 位宽度里做，硬件输出的就只有乘积的低 32 位——高位不是”算错了”，而是**根本没算**（那部分电路不存在）。`mul` 要的恰好是低 32 位，所以它不在乎；`mulh` / `mulhu` / `mulhsu` 要的是高位，所以必须先扩展。

1.4.2 扩展方式错了会怎样

以 `mulh` 为例，操作数 `a = 0xFFFF_FFFF` 的补码值是 -1。

做法	送进乘法器的 64 位值	结果
零扩展（补 0）	4294967295	算的是 $4294967295 \times b$ ，不是 <code>mulh</code> 要的数
符号扩展（复制符号位）	-1	算的是 $-1 \times b$ ，正确

所以”先扩展”的关键不是”补几个 0”，而是**按这条指令的解释方式、把值原样搬进 64 位**。`mulh` 要符号扩展，`mulhu` 要零扩展，`mulhsu` 要一个符号一个零——指令名里的 `s/u` 就是给这两个操作数各指定一种扩展方式。

1.4.3 为什么 mul 可以不扩展 (CS:APP 2.3.5)

CS:APP 2.3.5 节的结论：对两个 w 位位模式，先按无符号解释相乘、再按补码解释相乘，两者截断到 w 位后完全一样（位级一模一样）。用 1.2.1 的那条公式就能证明：

```
a_s = a_u - a31 * 2^32          // 下标 u 表示按无符号解释，s 表示按补码解释
b_s = b_u - b31 * 2^32

a_s * b_s = a_u * b_u - 2^32 * (a31 * b_u + b31 * a_u) + 2^64 * a31 * b31
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                这两项都含因子 2^32                                这一项含因子 2^64
```

只要截断到 32 位，含 2^{32} 和 2^{64} 的项统统变成 0，于是

```
a_s * b_s ≡ a_u * b_u (mod 2^32)    // 低 32 位与解释方式无关
```

所以 `mul`（只取低 32 位）不需要你说明符号性，硬件上也只需要一个乘法器。但低 32 位相同不等于整个 64 位乘积相同——差别全都跑到高 32 位去了。

1.4.4 高 32 位差多少：修正项

还是上面那个展开式，这次不截断，而是取高 32 位（ 2^{64} 那一项超出 64 位，被丢掉）：

```
mulh 的高 32 位 = mulhu 的高 32 位 - (a31 * b_u + b31 * a_u) (mod 2^32)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                修正项：两个操作数最高位分别"作梗"的代价
```

这条式子把三件事一次说清了：

1. 两个操作数最高位都是 0 时（都是正数/小无符号数），修正项为 0 → `mulh` 与 `mulhu` 结果相同；
2. 只要有一个操作数最高位是 1，高 32 位就会不同；
3. “先扩展再相乘”只是在实现同一个修正——差别永远只出现在高 32 位。

1.4.5 手算：先用 4 位玩具模型

把位宽缩到 4 位，运算规则一模一样，但能手算（ $w=4$ ，两个 4 位数的乘积要 8 位，正好分成高 4 位和低 4 位）：

```
a = 1111b    b = 0010b
按无符号解释：15 × 2 = 30 = 0001_1110b → 高 4 位 0001，低 4 位 1110
按补码解释：-1 × 2 = -2 = 1111_1110b → 高 4 位 1111，低 4 位 1110
```

低 4 位都是 1110b（对应 `mul` 的不在乎符号），高 4 位一个是 0001、一个是 1111。用修正项验证： $a_{31}=1$, $b_{31}=0$, $a_u=15$, $b_u=2$ → 修正项 = $1 \times 2 + 0 \times 15 = 2$, $0001 - 0010 = -1 = 1111b$ ✓

1.4.6 换成 32 位看一遍

取同一对 32 位操作数 $a = 0xFFFF_FFFF$ 、 $b = 0x0000_0002$ ，只换解释方式：

解释方式（对应指令）	数学乘积	64 位乘积	高 32 位	低 32 位
无符号 × 无符号 (mulhu)	8589934590	0x0000_0001_FFFF_FFFE	0x0000_0001	0xFFFF_FFFE
有符号 × 有符号 (mulh)	-2	0xFFFF_FFFF_FFFF_FFFE	0xFFFF_FFFF	0xFFFF_FFFE

低 32 位相同（所以 `mul` 两种写法都对），高 32 位相差 $0xFFFF_FFFF - 0x0000_0001 = -2$ ，正好等于修正项 $a_{31} \times b_u + b_{31} \times a_u = 1 \times 2 + 0 = 2$ 。

再看三条 `h` 指令在**同一对操作数**上的表现（这是自测题的素材）：

```
a = 0xFFFF_FFFF  b = 0x8000_0000
mulh   : -1 × (-2147483648) = 2147483648 = 0x0000_0000_8000_0000 高 32 位 0x0000_0000
mulhsu : -1 × 2147483648    = -2147483648 = 0xFFFF_FFFF_8000_0000 高 32 位 0xFFFF_FFFF
mulhu   : 4294967295 × 2147483648 = 0x7FFF_FFFF_8000_0000      高 32 位 0x7FFF_FFFF
```

加粗记一句：三条指令的差别，全都在”乘的是哪个值”上；起手式永远是”把值搬进 64 位”。

1.5 回到 Verilog: 怎么把”先扩展”写出来

1.5.1 隐式写法：靠上下文位宽 + 类型标记

Verilog/SystemVerilog 的表达式位宽由上下文决定：赋值号左边是 64 位，这个 64 位会向上传播给 `*`，两个 32 位操作数会被扩展成 64 位再相乘；而怎么扩展由操作数的类型标记决定——无符号类型补 0，有符号类型复制最高位。`$signed()` 的作用就是”把这个表达式标记成有符号”，它不改变任何位：

```
logic [31:0]      a, b;
logic signed [31:0] sa, sb;
assign sa = a;           // 只是换个类型标记，位模式没变
assign sb = b;

logic [63:0]      uprod;   assign uprod   = a * b;           // 两个操作数都零扩展
logic signed [63:0] sprod;  assign sprod   = sa * sb;        // 两个操作数都符号扩展
logic signed [63:0] sprod_su; assign sprod_su = sa * $signed({1'b0, b}); // a 符号扩展、b 零扩展
```

于是：

```
mulh   = sprod[63:32];
mulhsu = sprod_su[63:32];
mulhu  = uprod[63:32];
```

`$signed({1'b0, b})` 是 `mulhsu` 里最容易看不懂的一处：`{1'b0, b}` 拼出 33 位，最高位恒为 0，所以它的值就是 `b` 的无符号值；再标记成有符号类型，扩展到 64 位时补的是符号位 (0)——**等价于零扩展**，正好是”有符号 × 无符号”里 `b` 该有的待遇。直接写 `sa * $signed(b)` 就会把 `b` 也当成补码数（`0x8000_0000` 会变成 -2^{31} 而不是 2^{31} ），结果就错了。

1.5.2 显式写法：真的先扩展、再相乘

如果不放心”隐式扩展”，把扩展写出来，代码和 1.3 节一一对应：

```

logic [63:0]      a_u = {32'b0, a};           // 零扩展
logic [63:0]      b_u = {32'b0, b};
logic signed [63:0] a_s = {{32{a[31]}}, a};   // 符号扩展：把符号位复制 32 次
logic signed [63:0] b_s = {{32{b[31]}}, b};

assign uprod = a_u * b_u;                     // = a * b
assign sprod = a_s * b_s;                     // = sa * sb

```

两种写法综合出来的电路含义相同：位宽是 64 的乘法，操作数已经按正确的值扩展好了。

1.5.3 和 C 语言对照（CS:APP 2.3.4 的例子）

“先扩展还是先相乘”在 C 里同样存在，因为 C 的 `int` 乘法是 32 位、结果会被截断：

语言	先扩展（值不变）再相乘	先相乘再扩展（高位丢了）
C	<code>long long p = (long long)a * b;</code>	<code>long long p = a * b;</code>
Verilog	<code>assign sprod = \$signed(a) * \$signed(b);</code>	<code>assign sprod = \$signed(a * b);</code>

右列在 C 里是”先按 32 位乘、截断，再把结果扩展成 64 位”；在 Verilog 里 `$signed(a * b)` 则是”先把无符号 32×32 乘积算完，再把它标记成有符号”——**乘法本身仍然是无符号的**。别把 `$signed` 当成”切换成有符号乘法”的开关，它只影响**扩展规则**，而且要写在操作数上、不能写在乘积上。

1.5.4 常见错误对照

写法	实际发生的事
<code>assign sprod = a * b;</code> (a、b 无符号)	算的是无符号乘积 → <code>mulh</code> 位置拿到 <code>mulhu</code> 的高位
<code>assign sprod = \$signed(a * b);</code>	无符号乘积再被标记成有符号 → 同上，高位错
<code>assign uprod = sa * sb;</code>	有符号乘积 → <code>mulhu</code> 位置拿到 <code>mulh</code> 的高位
<code>logic signed [31:0] p = sa * sb;</code>	左值只有 32 位 → 乘积的高 32 位根本没算
<code>sa * \$signed(b)</code> (本意是 <code>mulhsu</code>)	变成有符号 × 有符号 → <code>b</code> 最高位为 1 时高位错

1.5.5 进阶：不扩展也能算高位（选读）

真实硬件常常只有一个”无符号 32×32 → 64 位”乘法器，此时不必再放一个 64 位乘法器，而是用 1.4.4 的修正项补一下：

```

logic [63:0] p    = {32'b0, a} * {32'b0, b}; // 无符号乘积（大多数乘法器本来就有）
logic [31:0] corr = (a[31] ? b : 32'b0) + (b[31] ? a : 32'b0);
// mulhu: p[63:32]
// mulh  : p[63:32] - corr
// mulhsu: p[63:32] - (a[31] ? b : 32'b0)      // b 无符号，所以只有 a 的符号位需要修正

```

看懂了这一节，就说明”为什么必须先扩展”已经从口诀变成了可以自己推导的结论：扩展只是把值搬进更宽的位宽的一种实现方式，真正不可省的是”乘法要按正确的值来算”。

1.6 三个必须按规范处理的边界

① 高位乘法要先扩展再相乘。这是 1.2 ~ 1.5 节的结论：`mulh` / `mulhsu` / `mulhu` 取的是同一个 64 位乘积的不同解释方式下的高位。对应的 Verilog 写法：

```

assign uprod    = a * b;           // 两个都零扩展 → mulhu
assign sprod    = sa * sb;         // 两个都符号扩展 → mulh
assign sprod_su = sa * $signed({1'b0, b}); // 符号 × 零扩展 → mulhsu

```

② 除零有明确规定（不能依赖硬件的”未定义行为”）：

- `div` / `divu` 返回全 1（即 -1）；
- `rem` / `remu` 返回被除数本身。

③ 唯一的有符号溢出：`INT_MIN` / -1。

- `div` 返回 `INT_MIN` (0x8000_0000)；
- `rem` 返回 0。

因为这三种情况都存在，代码里必须先用 `if` 挡住，再去算 `a / b`；直接写 `y = a / b` 在除零时会得到 X。

1.7 组合实现 vs 多周期实现

先把名字说清楚：MDU 是 **Multiply-Divide Unit（乘除法单元）** 的缩写，指”负责乘法和除法的那块执行单元”，与 ALU（算术逻辑单元 Arithmetic Logic Unit）、LSU（访存单元 Load-Store Unit）是同一套命名习惯。它不是 RISC-V 规范里的术语，各家的拆分方式也不一样：上游 CoralNPU 就把它拆成两个单元——`Mlu.scala`（Multiply Unit，只做乘法）和 `Dvu.scala`（Divide Unit，只做除法）；本课为了把 8 条指令的语义放在一起讲，合并成一个 `mdu.sv`。

本课把 MDU 做成**组合电路**（一拍出结果）。真实处理器通常做成多周期流水线除法器（每拍移位一次，32 拍完成），因为组合除法器的关键路径很长。

练习：上游 CoralNPU 的 `hdl/chisel/src/coralnpu/scalar/Mlu.scala` 与 `doc/microarch/mlu.md` 就是乘法单元的实现与接口说明——对比一下：它在流水线的哪一级？延迟几拍？除法那边再看 `scalar/Dvu.scala`：它每拍只算 1 位，属于典型的”多周期执行单元”（本课的 L02 LSU 也是这类单元的现成例子）。

第2章 CSR 与 Zicsr

2.1 CSR 是什么：机器的控制面板

CSR = **Control and Status Register**（控制状态寄存器）。和通用寄存器对比着记最省事：**GPR 装的是“数据”，CSR 装的是“机器自己的状态和控制开关”**。

	通用寄存器堆	CSR
装什么	程序的数据：临时变量、地址、计数器……	机器的状态与控制：特权级、异常信息、中断开关、性能计数器……
有多少	32 个（x0 – x31）	12 位地址空间，最多 4096 个
谁能访问	几乎所有指令都能把它当源/目的操作数	只有 Zicsr 的 6 条指令
在内存地址空间里吗	不在	也不在 ：独立的 CSR 地址空间，lw / sw 够不着
主要使用者	编译器生成的代码	固件（OpenSBI）、操作系统内核、调试器、性能分析工具

三个直接推论：

- **不能用 lw / sw 读写 CSR**——不是硬件偷懒，而是 CSR 根本不在内存映射里；
- **CSR 是架构状态**：操作系统做上下文切换时，除了 32 个 GPR，还要把一部分 CSR 一起保存/恢复（比如异常入口、当前特权级、中断使能、页表基址）；
- **CSR 是软件和硬件之间的控制接口**：硬件把“发生了什么”写进去（异常原因、计数器、状态位），软件把“接下来怎么做”写进去（中断使能、异常入口地址、页表基址）。

CS:APP 从软件侧讲了同一件事，可以对着看：

疑问	CS:APP（第3版）
异常/中断为什么能打断程序？内核怎么接管？	8.1 异常控制流、8.1.2 异常的类别（中断/陷阱/故障/终止）
进程切换时到底保存和恢复了什么？	8.4 进程控制
页表基址、地址翻译由谁控制？为什么需要特权级？	9.6 地址翻译、9.7 案例研究（Intel Core i7 / Linux）

2.2 CSR 都在管什么

规范里的 CSR 有几百个，按职责分类就清楚多了（本课实现的只有打✓的几项）：

职责	典型 CSR	本课
异常入口与现场	mtvec、mepc、mcause、mtval	✓
中断使能与中断源	mstatus.MIE、mie、mip	✗（mstatus 只当普通寄存器）
特权级与陷阱嵌套	mstatus.MPP/MPIE、medeleg、mideleg	✗
内存保护	pmpcfg0...、pmpaddr0...	✗
地址翻译	satp（S 级）、hvatp（H 级）	✗
计数器	mcycle、minstret、cycle、instret、hpmcounter*	✗
身份与能力发现	misa、mvendorid、marchid、mimpid、mhartid	✓（misa、mhartid，只读）
调试	dcsr、dpc、tselect、tdata1-3（0x7A0-0x7BF）	✗
厂商自定义	自定义地址段（见 2.3 节）	✗（上游 CoralNPU 用内存映射的 CSR 外壳，见 L04）

两个值得知道的具体例子：

- **misa 是”能力清单”**：软件启动时读它，就知道这颗核支持哪些扩展（0x4000_1100 表示 RV32 + I + M）。Linux/OpenSBI 这类固件真的会去探测它，所以它必须可读、且读出来的位要和硬件实际能力一致。
- **mhartid 是”我是谁”**：多核系统里每个 hart 的编号，启动代码靠它决定“我是 0 号核，我来初始化内存；其余的核先等着”。

2.3 CSR 地址不是随便编的：12 位里藏了两件事

CSR 地址固定 12 位，高 4 位是**编码**，不是名字的一部分（特权规范第 2 章）：

- csr[11:10]：00 / 01 / 10 = 可读可写；11 = 只读；
- csr[9:8]：最低能访问它的特权级：00 = U、01 = S、10 = H、11 = M；
- 于是地址的第二个十六进制位就告诉你它属于哪一级（0x0xx 用户级、0x1xx S 级、0x3xx M 级……），而 0xC00-0xFFFF 这一段（最高两位是 11）一定是只读的。

用这两条规则”拆”一遍本课用到的地址：

```

0x300 mstatus : csr[11:10]=00 可读写，csr[9:8]=11 机器级 → M 级可读写
0x341 mepc    : 同上                                     → M 级可读写
0xF14 mhartid : csr[11:10]=11 只读，csr[9:8]=11 机器级 → M 级只读
0xC00 cycle   : csr[11:10]=11 只读，csr[9:8]=00 用户级 → 用户级只读

```

地址空间的整体划分（简化版）：

地址范围	归属	访问
0x000–0x0FF、0x400–0x4FF	用户级：标准段	读写
0x100–0x1FF、0x500–0x5FF	S 级：标准段	读写
0x200–0x2FF、0x600–0x6FF	H 级：标准段	读写
0x300–0x3FF、0x700–0x77F	M 级：标准段	读写
0x800–0x8FF、0x9C0–0x9FF、 0xAC0–0xAFF、0x7C0–0x7FF	各级：自定义段	读写
0xC00–0xFFFF	各级：只读段（标准 + 自定义）	只读
0x7A0–0x7BF	调试专用	读写（其中 0x7B0–0x7BF 只有调试模式能看）

（细节以特权规范第 2 章那张表为准；这里只保留理解地址规律所需的部分。）

自定义段是给硬件厂商准备的：规范承诺“自定义地址不会被未来的标准扩展重新定义”，所以加速器、性能监控、调试口这些非标准功能都往这里放。做 NPU 时想给 CPU 加一组“配置矩阵引擎”的寄存器，这就是常规去处——本课不做，但你在 L04 会看到另一种常见做法：把控制寄存器做成内存映射的外壳（CoreAxiCSR）。

访问违规的后果（规范里的硬性规定）：

- 特权级不够 → **非法指令异常**（不是返回 0！）；
- 写只读 CSR → **非法指令异常**；
- 读写 CSR 里的**只读位** → 写入被忽略，不报错；
- 访问不存在的 CSR → 行为保留（本课约定按 0 读、写无效，真实处理器一般报非法指令）。

本课的简化：所有程序都跑在 M 模式，`csr_file` 不做特权检查，写只读寄存器只是没有效果。

2.4 Zicsr 六条指令

Zicsr 扩展提供 6 条指令（`csrrw` 是“读旧值 + 写新值”，另外两条是“读旧值 + 改几位”）：

指令	行为	读?	写?
<code>csrrw rd, csr, rs1</code>	读旧值 $\rightarrow$ rd ; 写 rs1 $\rightarrow$ csr	✓	✓ (总是写)
<code>csrrs rd, csr, rs1</code>	读旧值 $\rightarrow$ rd ; csr $\setminus$ = rs1	✓	置位
<code>csrrc rd, csr, rs1</code>	读旧值 $\rightarrow$ rd ; csr &= ~rs1	✓	清位
<code>csrrwi/csrrsi/csrrci</code>	同上, rs1 换成 5 位立即数	✓	同上

2.4.1 为什么 csrrs rd, csr, x0 不能写

规范规定：`csrrs / csrrc`（以及立即数形式）在 **rs1 = x0** 时完全不写 CSR。理由有两条，都和“写”有关：

1. 标准 CSR 读没有副作用，写可能有副作用。`csrrs rd, csr, x0` 是编译器生成 `csrr rd, csr`（纯读）时用的编码，如果它也执行“读改写”，就会凭空触发一次写副作用；
2. 对只读 CSR 的写会触发非法指令异常。如果 `csrrs rd, csr, x0` 算写，那么连“看一眼 mhartid”“都做不到了”。

顺带把另外两种情况也记住（实现时千万别搞反）：

- `csrrw rd, csr, x0` 是**写 0**（会写！），常用来清一个 CSR；
- `csrrw x0, csr, rs1` 是**只写不读**：rd = x0 时指令不读 CSR，因此也不会触发读副作用。

2.4.2 汇编伪指令

汇编器给常用写法提供了别名，读测试程序时会遇到：

伪指令	实际编码
<code>csrr rd, csr</code>	<code>csrrs rd, csr, x0</code> （只读）
<code>csrw csr, rs1</code>	<code>csrrw x0, csr, rs1</code> （只写）
<code>csrs csr, rs1 / csrrc csr, rs1</code>	<code>csrrs / csrrc</code> （不关心旧值）
<code>csrwi / csrsi / csrrci csr, uimm</code>	立即数形式

本课测试程序里的 `csrw mtvec, t0`、`csrr t2, mepc` 就是这两条。

2.4.3 三个容易忽略的细节

- **原子性**：`csrrs / csrrc` 是“读一改一写”一条完成，中间不会被中断或其它 hart 插进来。这是规范故意设计的——否则“置位某个中断使能位”这种操作会有竞态。

- **立即数只有 5 位**（0–31）：看起来很小，但 CSR 里需要经常置位/清零的都是低位字段（比如 `mstatus` 的中断使能位在 bit 3），`csrsi mstatus, 8` 一条指令就够。
- **顺序性**：同一条 hart 上，CSR 访问按程序序可见——读返回的是“这条指令之前”的值，写对“后面的指令”生效。这正是 2.8 节那个旁路要保证的东西。

2.5 字段的读写规则：RO / WARL / WLRL

规范给 CSR 里的**字段**（不是整个寄存器）定了三种约定：

约定	含义	例子
RO（只读）	写入被忽略；往只读 CSR 写会触发非法指令异常	<code>mhartid</code> 、 <code>mvendorid</code> 、 <code>cycle</code>
WARL（Write Any, Read Legal）	随便写，硬件只保证读回来的是 合法值	<code>misa</code> （不能让软件谎报扩展）、 <code>mtvec.MODE</code> （只支持 0/1）
WLRL（Write Legal, Read Legal）	只有写合法值才有保证	某些性能计数器的事件选择字段

这些约定的意义是：**软件不能把硬件的状态机写坏**。最典型的例子是 `mtvec` 的低 2 位：写 0 表示 Direct（所有陷阱都跳到 `BASE`），写 1 表示 Vectored（中断跳到 `BASE + 4 × 原因码`），写 2、3 是保留值——真实处理器会把保留值“折回”一个合法值（WARL），而不是让 CPU 跳到一个莫名其妙的地址。本课为简单起见把 `mtvec` 当普通寄存器存，也不实现 Vectored 模式。

2.6 一次真实的异常：CSR 各司其职

把第 3 章的流程换成 CSR 的视角（括号里是本课的简化）：

```

进入异常 —— 硬件执行，全是 CSR 写：
mepc      ← 触发异常的指令地址
mcause    ← 原因码（最高位 = 1 表示中断，0 表示同步异常）
mtval     ← 附加信息（非法指令编码、出错地址……；本课写 0）
mstatus.MPIE ← mstatus.MIE      把“中断开着吗”压栈
mstatus.MIE  ← 0                处理程序里先关中断，避免嵌套
mstatus.MPP ← 当前特权级        记住从哪一级来的
pc        ← mtvec              Direct 模式；Vectored 模式是 mtvec + 4 × 原因码

返回 —— mret 执行，仍然是 CSR 读写：
pc        ← mepc
mstatus.MIE ← mstatus.MPIE      恢复中断使能
mstatus.MPIE ← 1
特权级    ← mstatus.MPP        回到原来的特权级
  
```

（本课只实现 `mtvec` / `mepc` / `mcause` / `mtval`，`mstatus` 当普通寄存器，`mret` 只做 `pc ← mepc` ——见 2.7 节。）

`mcause` 的最高位是**中断标志**，所以同一个寄存器既能表示 `ecall`（值 11），也能表示“机器定时器中断”（值 `0x8000_0007`）——第 3.3 节那张表只列了低位原因码。

看完这一串操作，2.1 节那句”CSR 是架构状态”就落地了：处理器「在哪个特权级、中断开没开、刚才为什么停下来的」全都记在 CSR 里，所以上下文切换必须连它们一起保存。

2.7 本课实现的最小集合

地址	名字	权限	用途
0x300	mstatus	读写	机器模式状态（本课只当普通寄存器用）
0x301	misa	只读	支持的扩展：0x4000_1100 = RV32 + I + M
0x305	mtvec	读写	异常入口地址
0x340	mscratch	读写	给异常处理程序暂存数据
0x341	mepc	读写	触发异常的指令地址
0x342	mcause	读写	异常原因码
0x343	mtval	读写	附加信息（本课写 0）
0xF14	mhartid	只读	硬件线程号（0）

读一个未实现的地址按 0 处理、写它没有效果（本课的简化约定；真实处理器通常会报非法指令）。

两点取舍，对照 2.5 节看：

- **misa**：真实规范里它是 **WARL**（可以写，但硬件只允许关掉自己实现的扩展位）；本课直接做成整个只读，读回 0x4000_1100。
- **mstatus**：真实硬件的 **MIE**（bit 3）、**MPIE**（bit 7）、**MPP**（bit 12:11）是中断与特权级逻辑的核心；本课按 2.6 节的简化把它当普通寄存器，**mret** 也不动特权级。

2.8 CSR 也有数据冒险（本课踩过的坑）

```

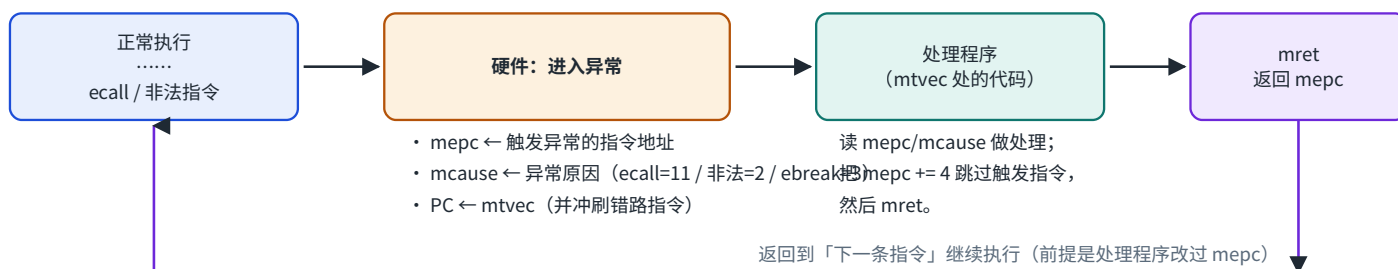
csrw mepc, t2      # 在 WB 级才真正写进 CSR
mret               # 下一条就在 EX 级读 mepc —— 读到的是旧值！

```

这和寄存器堆的 RAW 冒险是一回事，处理办法也一样：**EX 级读 CSR 时，如果 MEM/WB 里有对同一个 CSR 的待提交写，就用它的值（CSR 旁路）**。本课的核心代码里已经实现了这条旁路（`ex_csr_rdata_f` 那段），你在读代码时可以留意。

第3章 异常：进入与返回

异常的进入与返回：ecall → 处理程序 → mret



本课实现的最小 CSR 集合

0x300 mstatus	读写	机器状态 (本课当普通寄存器)
0x301 misa	只读	0x4000_1100 = RV32 + I + M
0x305 mtvec	读写	异常入口地址
0x340 mscratch	读写	给处理程序暂存数据
0x341 mepc	读写	触发异常的指令地址
0x342 mcause	读写	异常原因码

关键细节：mepc 存的是「触发异常的那条指令」，所以处理程序必须自己 +4 才能继续往下跑

异常流程

3.1 三个概念

- **异常** (exception)：由指令自己引起 (ecall、非法指令、访存错误……)；
- **mtvec**：异常处理程序的入口地址；
- **mret**：从异常返回，回到 mepc。

3.2 完整的流程

- ① 软件在启动时设置 mtvec (异常入口)
csw mtvec, t0
- ② 指令触发异常 (例如 ecall, mcause = 11)
硬件做三件事：
mepc ← 触发异常的指令地址 (本例是 ecall 自己的地址)
mcause ← 11
PC ← mtvec
同时冲刷异常指令后面的错路指令 (和分支冲刷一样)
- ③ 异常处理程序运行，读 mepc/mcause，做该做的事
csrr t2, mepc
csrr t3, mcause
addi t2, t2, 4 ← 关键：跳过触发异常的指令
csw mepc, t2
- ④ mret 返回
PC ← mepc ← 已经变成“下一条指令”的地址

为什么要 `mepc += 4`？因为 `mepc` 存的是触发异常的指令的地址。如果异常处理程序什么都不改就 `mret`，会重新执行那条指令 → 再次异常 → 死循环。（这正是检查器在异常用例超时时会提示你的那句话。）

3.3 常见的 mcause 取值

值	含义	本课会遇到的
2	非法指令	✓（ <code>.word 0</code> ）
3	断点（ <code>ebreak</code> ）	✓（启动代码的失败路径）
11	来自 M 模式的环境调用（ <code>ecall</code> ）	✓

更完整的列表见 RISC-V 特权规范第 3.1.15 节（Trap Cause Codes）。

3.4 和上游的关系

上游 CoralNPU 有一整套异常机制：`FaultManager.scala` 收集各执行单元的错误，`Csr.scala` 维护 CSR，默认的异常处理程序（`toolchain/crt/coralnpu_gloss.cc`）只是把 `ebreak` 打出来。本课实现的是它的最小子集：

本课	上游
<code>mepc/mcause/mstatus/mtvec</code>	完整的机器模式 CSR（含中断、计数器、特权级）
<code>ecall</code> / 非法指令	访存错误、指令错误、总线错误……
同步异常	同步 + 异步（中断、定时器、软件中断）

第4章 自测题

1. `0xFFFF_FFFF` 在 `mulh` 与 `mulhu` 里分别被当成哪个值？（1.2 节）
2. `mulh` 与 `mulhu` 对同一对操作数为什么可能不同？用修正项算一遍并举例。（1.4 节）
3. 用 4 位玩具模型手算 $1111b \times 0010b$ 的两种”高位乘积”，并验证低 4 位相同、高 4 位不同。（1.4.5 节）
4. 为什么 `mul` 可以不管符号性？它的哪一部分与解释方式无关？（1.4.3 节）
5. `mulhsu` 里 `b` 为什么是零扩展？写成 `$signed(b)` 会在什么时候出错？（1.5.1 节）
6. 为什么 `assign sprod = $signed(a * b);` 不能代替 `$signed(a) * $signed(b)`？（1.5.3 节）
7. `div x1, x2, x0` 的结果是什么？`rem x1, x2, x0` 呢？
8. `INT_MIN / -1` 的结果是什么？为什么它不算”错误”？
9. 为什么不能直接写 `y = a / b`，而要先用 `if` 判断除零？
10. CSR 地址只有 12 位，怎么从中看出”能不能写””谁才能访问”？（提示：`csr[11:10]`、`csr[9:8]`）
本课的 `mhartid = 0xF14` 是怎么编码出来的？
11. `csrrs rd, mstatus, x0` 会修改 `mstatus` 吗？为什么这么规定？`csrrw x0, mstatus, t0` 又会不会读 `mstatus`？
12. WARL 是什么意思？举一个例子，说明为什么需要这种约定。
13. 真实硬件进入异常时对 `mstatus` 做了哪三件事？`mret` 返回时又做了什么？（2.6 节）
14. `mcause` 的最高位是什么？为什么 `0x8000_0007` 和 `7` 表示的是不同的事件？
15. 异常发生后 `mepc` 里是什么？谁负责把它改成”下一条指令”？
16. 为什么 CSR 读写也需要旁路？（提示：`csrw mepc` 紧跟 `mret`）
17. 如果异常处理程序忘了 `csrw mepc`，程序会怎样？（无限触发同一个异常）

第 5 章 参考资料

- CS:APP 《深入理解计算机系统》（原书第 3 版）第 2 章「信息的表示和处理」：2.2.2 / 2.2.3（位模式的两种解释）、2.2.6（零扩展与符号扩展）、2.2.7（截断）、2.3.4 / 2.3.5（无符号乘法与补码乘法）；第 3 章 3.5.5 节讲了 x86-64 里 `mulq / imulq` 如何产生 128 位乘积——和 `mulhu / mulh` 是同一件事。配套习题（乘法溢出检测、用有符号乘法实现无符号乘法）建议一起做。
- CS:APP 第 8 章「异常控制流」（8.1 异常、8.1.2 异常的类别）与第 9 章 9.6 / 9.7（地址翻译与特权级）：从操作系统视角回答“内核为什么需要这些寄存器”，正好和本章的硬件视角互补。
- RISC-V 非特权规范：M 扩展第 7 章、Zicsr 第 9 章 <https://riscv.org/technical/specifications/>
- RISC-V 特权规范：第 2 章（CSR 地址映射约定：读写权限、特权级编码、自定义地址段）、第 3 章（机器模式 CSR： `mstatus` 的字段、 `mtvec` 的 Direct/Vectored、 `mepc / mcause / mtval` 与 `mret` 的完整语义）
- 本仓库 [doc/microarch/mlu.md](#)：上游乘法单元的接口与流水
- 本仓库 `hdl/chisel/src/coralnpu/scalar/Csr.scala`、 `FaultManager.scala`：上游实现（做完再看）